

# Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Apuntes de SQL (PostgreSQL) — Guía para las 20 consultas sobre Northwind</b> | <b>2</b>  |
| 1. WHERE, BETWEEN, IN, LIKE . . . . .                                           | 2         |
| 2. NULL y COALESCE() . . . . .                                                  | 2         |
| 3. Funciones de agregación y ROUND() . . . . .                                  | 3         |
| 4. GROUP BY y HAVING . . . . .                                                  | 3         |
| 5. INNER JOIN, alias de tabla y USING . . . . .                                 | 3         |
| 6. LEFT JOIN (unión externa) . . . . .                                          | 4         |
| 7. SELF JOIN (autounión) . . . . .                                              | 5         |
| 8. CROSS JOIN (producto cartesiano) . . . . .                                   | 5         |
| 9. FULL JOIN (unión externa completa) . . . . .                                 | 6         |
| 10. UNION / UNION ALL . . . . .                                                 | 6         |
| 11. EXCEPT e INTERSECT . . . . .                                                | 7         |
| 12. Semi join y anti join (EXISTS / NOT EXISTS / NOT IN) . . . . .              | 7         |
| 13. Subconsulta escalar en WHERE y en SELECT . . . . .                          | 8         |
| 14. Subconsulta en FROM (tabla derivada) . . . . .                              | 8         |
| 15. Subconsultas correlacionadas . . . . .                                      | 9         |
| 16. CTE / WITH (expresiones de tabla común) . . . . .                           | 9         |
| 17. CASE WHEN (condicional en SQL) . . . . .                                    | 10        |
| 18. NTILE() — segmentación en tramos . . . . .                                  | 11        |
| 19. RANK(), DENSE_RANK(), ROW_NUMBER(), PARTITION BY . . . . .                  | 11        |
| 20. Marcos de ventana (ROWS BETWEEN), acumulados y LAG() . . . . .              | 12        |
| 21. Pivotado manual (FILTER / CASE WHEN) y ROLLUP . . . . .                     | 12        |
| 22. Funciones de texto . . . . .                                                | 13        |
| 23. Funciones de fecha (EXTRACT, DATE_TRUNC) . . . . .                          | 13        |
| 24. Cast a numeric en cálculos monetarios . . . . .                             | 14        |
| 25. Checklist rápido de técnicas por pregunta . . . . .                         | 14        |
| <b>Ejercicios resueltos</b>                                                     | <b>15</b> |
| Pregunta 1 — Catálogo comercial activo . . . . .                                | 15        |
| Pregunta 2 — Concentración geográfica de la cartera . . . . .                   | 15        |
| Pregunta 3 — Alerta de reposición . . . . .                                     | 15        |
| Pregunta 4 — Ficha completa de producto . . . . .                               | 16        |
| Pregunta 5 — Detalle valorizado de un pedido . . . . .                          | 16        |
| Pregunta 6 — Ranking de categorías por facturación . . . . .                    | 16        |
| Pregunta 7 — Clientes sin actividad comercial . . . . .                         | 17        |
| Pregunta 8 — Organigrama de la fuerza de ventas . . . . .                       | 17        |
| Pregunta 9 — Rejilla de cobertura categoría × año . . . . .                     | 17        |
| Pregunta 10 — Mapa de países: clientes frente a proveedores . . . . .           | 17        |
| Pregunta 11 — Directorio unificado de contactos . . . . .                       | 18        |
| Pregunta 12 — Mercados con desequilibrio . . . . .                              | 19        |
| Pregunta 13 — Clientes que nunca han comprado pescado . . . . .                 | 19        |
| Pregunta 14 — Productos por encima de la media . . . . .                        | 19        |
| Pregunta 15 — Ticket medio por cliente . . . . .                                | 20        |
| Pregunta 16 — El producto más caro de cada categoría . . . . .                  | 20        |
| Pregunta 17 — Segmentación ABC de la cartera de clientes . . . . .              | 20        |
| Pregunta 18 — Los tres productos más vendidos de cada categoría . . . . .       | 21        |
| Pregunta 19 — Evolución mensual con acumulado y media móvil . . . . .           | 22        |
| Pregunta 20 — Cuadro de mando anual por categoría . . . . .                     | 22        |

# Apuntes de SQL (PostgreSQL) – Guía para las 20 consultas sobre Northwind

Formato de cada apunte: **Concepto** → **Explicación** → **Sintaxis básica** → **Ejemplo práctico**. Los ejemplos usan el esquema de Northwind (customers, orders, order\_details, products, categories, suppliers, employees) para que puedas adaptarlos directamente a las preguntas de la práctica.

---

## 1. WHERE, BETWEEN, IN, LIKE

**Explicación:** WHERE filtra filas **antes** de agrupar. BETWEEN comprueba un rango inclusivo. IN comprueba pertenencia a una lista. LIKE compara patrones de texto (% = cualquier secuencia, \_ = un carácter).

**Sintaxis básica:**

```
WHERE columna BETWEEN valor1 AND valor2
WHERE columna IN (valor1, valor2, valor3)
WHERE columna LIKE 'patron%'
```

**Ejemplo práctico:**

```
SELECT product_name AS producto,
       ROUND(unit_price::numeric, 2) AS precio
FROM products
WHERE discontinued = 0
   AND unit_price BETWEEN 10 AND 50
ORDER BY precio DESC;
```

---

## 2. NULL y COALESCE()

**Explicación:** NULL representa “ausencia de valor”. Cualquier comparación con NULL (=, >, etc.) devuelve NULL (ni verdadero ni falso), así que la fila **no aparece** en un WHERE normal sin avisar. COALESCE(a, b, ...) devuelve el primer valor no nulo de la lista; sirve para sustituir un NULL por un valor por defecto.

**Sintaxis básica:**

```
columna IS NULL
columna IS NOT NULL
COALESCE(columna, 'valor_por_defecto')
```

**Ejemplo práctico:**

```
SELECT c.company_name AS cliente,
       COALESCE(MAX(o.order_date)::text, 'SIN PEDIDOS') AS ultimo_pedido
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.company_name;
```

---

### 3. Funciones de agregación y ROUND()

**Explicación:** SUM, COUNT, AVG, MAX, MIN resumen varias filas en un solo valor. COUNT(\*) cuenta filas (incluidas las generadas por un LEFT JOIN con nulos); COUNT(columna) cuenta solo valores no nulos de esa columna — la diferencia es clave para detectar clientes sin pedidos. ROUND(valor::numeric, decimales) redondea; conviene castear a numeric en cálculos monetarios porque real/double precision acumulan errores de redondeo.

**Sintaxis básica:**

```
SELECT SUM(expresion), COUNT(columna), AVG(expresion), MAX(columna)
FROM tabla;
```

```
ROUND((unit_price::numeric) * quantity * (1 - discount::numeric), 2)
```

**Ejemplo práctico:**

```
SELECT COUNT(*) AS total_filas,
       COUNT(o.order_id) AS pedidos_reales,
       ROUND(SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)), 2) AS factur
FROM order_details od
JOIN orders o USING (order_id);
```

---

### 4. GROUP BY y HAVING

**Explicación:** GROUP BY agrupa filas que comparten valor en una o varias columnas, permitiendo aplicar funciones de agregación por grupo. HAVING filtra **después** de agrupar (sobre el resultado agregado); WHERE filtra antes y no puede usar funciones de agregación.

**Sintaxis básica:**

```
SELECT columna_agrupacion, COUNT(*) AS total
FROM tabla
GROUP BY columna_agrupacion
HAVING COUNT(*) >= 5
ORDER BY total DESC;
```

**Ejemplo práctico:**

```
SELECT c.country AS pais,
       COUNT(*) AS num_clientes,
       COUNT(DISTINCT c.city) AS num_ciudades
FROM customers c
GROUP BY c.country
HAVING COUNT(*) >= 5
ORDER BY num_clientes DESC;
```

---

### 5. INNER JOIN, alias de tabla y USING

**Explicación:** INNER JOIN combina filas de dos tablas que cumplen una condición de unión, descartando las que no tienen correspondencia en ambos lados. Los alias (c, o, p...) acortan la escritura y son obligatorios cuando una tabla aparece más de una vez. USING(columna) es un atajo de ON a.col = b.col que solo funciona cuando la columna de unión se llama igual en ambas tablas, y evita que salga duplicada en el resultado.

**Sintaxis básica:**

```

SELECT a.col1, b.col2
FROM tabla_a a
JOIN tabla_b b ON a.id = b.id;

-- equivalente si el nombre coincide:
SELECT a.col1, b.col2
FROM tabla_a a
JOIN tabla_b b USING (id);

```

### Ejemplo práctico:

```

SELECT p.product_name AS producto,
       cat.category_name AS categoria,
       s.company_name AS proveedor,
       s.country AS pais,
       s.city AS ciudad
FROM products p
JOIN categories cat ON cat.category_id = p.category_id
JOIN suppliers s ON s.supplier_id = p.supplier_id
WHERE s.country IN ('Italy', 'France', 'Spain')
ORDER BY s.country, p.product_name;

```

Unión de tres tablas con order\_details usando USING (mismos nombres de columna a ambos lados):

```

SELECT c.company_name AS cliente,
       o.order_date AS fecha_pedido,
       p.product_name AS producto,
       od.unit_price AS precio_unitario,
       od.quantity AS cantidad,
       od.discount AS descuento,
       ROUND(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric), 2) AS importe_linea
FROM orders o
JOIN customers c USING (customer_id)
JOIN order_details od USING (order_id)
JOIN products p USING (product_id)
WHERE o.order_id = 10248;

```

## 6. LEFT JOIN (unión externa)

**Explicación:** Devuelve **todas** las filas de la tabla izquierda, tengan o no correspondencia en la derecha; cuando no la hay, las columnas de la derecha salen NULL. Es la herramienta para detectar “lo que falta” (clientes sin pedidos, etc.), combinada normalmente con COALESCE o IS NULL.

### Sintaxis básica:

```

SELECT a.col, b.col
FROM tabla_a a
LEFT JOIN tabla_b b ON a.id = b.a_id;

```

### Ejemplo práctico:

```

SELECT c.company_name AS cliente,
       c.country AS pais,
       COUNT(o.order_id) AS num_pedidos,
       COALESCE(MAX(o.order_date)::text, 'SIN PEDIDOS') AS ultimo_pedido

```

```
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.company_name, c.country
ORDER BY num_pedidos ASC;
```

Anti-join equivalente con LEFT JOIN (clientes sin ningún pedido):

```
SELECT c.company_name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

## 7. SELF JOIN (autounión)

**Explicación:** Una tabla se une consigo misma para relacionar filas entre sí (típicamente jerarquías, como empleados que reportan a otros empleados). Es obligatorio usar dos alias distintos para poder referirse a “la misma tabla” como si fueran dos tablas diferentes.

**Sintaxis básica:**

```
SELECT e1.col, e2.col
FROM tabla e1
LEFT JOIN tabla e2 ON e1.jefe_id = e2.id;
```

**Ejemplo práctico:**

```
SELECT emp.first_name || ' ' || emp.last_name AS empleado,
       emp.title AS cargo,
       COALESCE(jefe.first_name || ' ' || jefe.last_name, 'DIRECCIÓN GENERAL') AS responsable,
       COALESCE(jefe.title, '') AS cargo_responsable
FROM employees emp
LEFT JOIN employees jefe ON emp.reports_to = jefe.employee_id;
```

## 8. CROSS JOIN (producto cartesiano)

**Explicación:** Combina **todas** las filas de una tabla con **todas** las filas de otra, sin condición de unión ( $n^{\circ}$  filas resultado =  $\text{filas\_A} \times \text{filas\_B}$ ). Se usa para generar “rejillas” completas de combinaciones (por ejemplo, categoría  $\times$  año), que luego se rellenan con un LEFT JOIN para que no falten combinaciones sin datos.

**Sintaxis básica:**

```
SELECT a.col, b.col
FROM tabla_a a
CROSS JOIN tabla_b b;
```

**Ejemplo práctico:**

```
WITH anios AS (
  SELECT generate_series(1996, 1998) AS anio
)
SELECT cat.category_name AS categoria,
       a.anio,
       COALESCE(SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)), 0) AS fact
FROM categories cat
```

```

CROSS JOIN anios a
LEFT JOIN products p ON p.category_id = cat.category_id
LEFT JOIN order_details od ON od.product_id = p.product_id
LEFT JOIN orders o ON o.order_id = od.order_id AND EXTRACT(YEAR FROM o.order_date) = a.anio
GROUP BY cat.category_name, a.anio
ORDER BY cat.category_name, a.anio;

```

---

## 9. FULL JOIN (unión externa completa)

**Explicación:** Devuelve todas las filas de ambas tablas, coincidan o no; donde no hay correspondencia, las columnas del otro lado salen NULL. Al seleccionar la columna de unión hay que usar COALESCE de ambos lados, porque puede venir nula por cualquiera de los dos.

**Sintaxis básica:**

```

SELECT COALESCE(a.clave, b.clave) AS clave, a.col, b.col
FROM tabla_a a
FULL JOIN tabla_b b ON a.clave = b.clave;

```

**Ejemplo práctico:**

```

WITH clientes_pais AS (
    SELECT country, COUNT(*) AS n FROM customers GROUP BY country
),
proveedores_pais AS (
    SELECT country, COUNT(*) AS n FROM suppliers GROUP BY country
)
SELECT COALESCE(cp.country, pp.country) AS pais,
       COALESCE(cp.n, 0) AS num_clientes,
       COALESCE(pp.n, 0) AS num_proveedores,
       CASE
           WHEN cp.n IS NOT NULL AND pp.n IS NOT NULL THEN 'AMBOS'
           WHEN cp.n IS NOT NULL THEN 'SOLO CLIENTES'
           ELSE 'SOLO PROVEEDORES'
       END AS tipo_presencia
FROM clientes_pais cp
FULL JOIN proveedores_pais pp ON cp.country = pp.country;

```

---

## 10. UNION / UNION ALL

**Explicación:** Apilan verticalmente los resultados de varias consultas con el mismo número de columnas, en el mismo orden y con tipos compatibles. UNION elimina duplicados (más lento); UNION ALL los conserva (más rápido, y es lo correcto cuando filas idénticas son legítimamente distintas, como contactos de orígenes distintos).

**Sintaxis básica:**

```

SELECT col1, col2 FROM tabla_a
UNION ALL
SELECT col1, col2 FROM tabla_b;

```

**Ejemplo práctico:**

```

SELECT 'CLIENTE' AS origen, UPPER(contact_name) AS contacto, company_name AS organizacion, city AS ciudad
FROM customers
UNION ALL
SELECT 'PROVEEDOR', UPPER(contact_name), company_name, city, country
FROM suppliers
UNION ALL
SELECT 'EMPLEADO', UPPER(first_name || ' ' || last_name), 'NORTHWIND TRADERS', city, country
FROM employees
ORDER BY origen, pais;

```

---

## 11. EXCEPT e INTERSECT

**Explicación:** Operadores de conjunto que comparan el resultado de dos consultas (mismo número y tipo de columnas). EXCEPT devuelve las filas del primer conjunto que **no** están en el segundo. INTERSECT devuelve las filas que están en **ambos**. Ambos eliminan duplicados automáticamente.

**Sintaxis básica:**

```

SELECT col FROM tabla_a
EXCEPT
SELECT col FROM tabla_b;

SELECT col FROM tabla_a
INTERSECT
SELECT col FROM tabla_b;

```

**Ejemplo práctico:**

```

-- a) países con clientes pero sin proveedores
SELECT country FROM customers
EXCEPT
SELECT country FROM suppliers
ORDER BY country;

-- b) países con clientes y proveedores a la vez
SELECT country FROM customers
INTERSECT
SELECT country FROM suppliers
ORDER BY country;

```

---

## 12. Semi join y anti join (EXISTS / NOT EXISTS / NOT IN)

**Explicación:** Un **semi join** (EXISTS) comprueba si existe al menos una fila relacionada, sin traer sus columnas. Un **anti join** (NOT EXISTS) comprueba lo contrario: que no existe ninguna. NOT EXISTS es más seguro que NOT IN porque, si la subconsulta de NOT IN devuelve algún NULL, el resultado completo queda vacío sin error visible.

**Sintaxis básica:**

```

WHERE EXISTS (SELECT 1 FROM tabla_b b WHERE b.col = a.col)
WHERE NOT EXISTS (SELECT 1 FROM tabla_b b WHERE b.col = a.col)

```

**Ejemplo práctico:**

```

SELECT c.company_name AS cliente,
       c.country AS pais,
       COUNT(o.order_id) AS pedidos_realizados
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o2
    JOIN order_details od ON od.order_id = o2.order_id
    JOIN products p ON p.product_id = od.product_id
    JOIN categories cat ON cat.category_id = p.category_id
    WHERE o2.customer_id = c.customer_id
      AND cat.category_name = 'Seafood'
)
GROUP BY c.company_name, c.country
ORDER BY pedidos_realizados DESC;

```

---

### 13. Subconsulta escalar en WHERE y en SELECT

**Explicación:** Una subconsulta **escalar** devuelve exactamente un valor (una fila, una columna) y se puede usar en cualquier sitio donde iría un valor literal: dentro de una comparación en WHERE, o como una columna más en SELECT.

**Sintaxis básica:**

```

WHERE columna > (SELECT AVG(columna) FROM tabla)

SELECT columna, (SELECT AVG(columna) FROM tabla) AS media
FROM tabla;

```

**Ejemplo práctico:**

```

SELECT product_name AS producto,
       ROUND(unit_price::numeric, 2) AS precio,
       ROUND((SELECT AVG(unit_price) FROM products WHERE discontinued = 0)::numeric, 2) AS precio_promedio,
       ROUND(unit_price::numeric - (SELECT AVG(unit_price) FROM products WHERE discontinued = 0)) AS diferencia
FROM products
WHERE discontinued = 0
  AND unit_price > (SELECT AVG(unit_price) FROM products WHERE discontinued = 0)
ORDER BY diferencia DESC;

```

---

### 14. Subconsulta en FROM (tabla derivada)

**Explicación:** Una consulta completa se coloca dentro del FROM como si fuera una tabla, permitiendo agregar en dos niveles (por ejemplo, primero calcular el importe por pedido y después promediar esos importes por cliente). En PostgreSQL **es obligatorio** darle un alias a la subconsulta.

**Sintaxis básica:**

```

SELECT alias.columna, AVG(alias.otra_columna)
FROM (
    SELECT ... FROM tabla ...
)

```

```
) AS alias
GROUP BY alias.columna;
```

### Ejemplo práctico:

```
SELECT cliente, pais,
       COUNT(*) AS num_pedidos,
       ROUND(SUM(importe_pedido), 2) AS importe_total,
       ROUND(AVG(importe_pedido), 2) AS ticket_medio
FROM (
  SELECT o.order_id,
         c.company_name AS cliente,
         c.country AS pais,
         SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS importe_pedido
  FROM orders o
  JOIN customers c USING (customer_id)
  JOIN order_details od USING (order_id)
  GROUP BY o.order_id, c.company_name, c.country
) AS pedidos_valorizados
GROUP BY cliente, pais
ORDER BY ticket_medio DESC
LIMIT 15;
```

## 15. Subconsultas correlacionadas

**Explicación:** Subconsulta que hace referencia a una columna de la consulta externa, por lo que conceptualmente se reevalúa fila a fila de la consulta externa. Sirve, por ejemplo, para comparar cada fila con el máximo/promedio de su propio grupo sin necesidad de un GROUP BY separado.

### Sintaxis básica:

```
SELECT a.col
FROM tabla_a a
WHERE a.valor = (SELECT MAX(b.valor) FROM tabla_b b WHERE b.grupo = a.grupo);
```

### Ejemplo práctico:

```
SELECT cat.category_name AS categoria,
       p.product_name AS producto,
       ROUND(p.unit_price::numeric, 2) AS precio,
       ROUND((SELECT AVG(p2.unit_price) FROM products p2 WHERE p2.category_id = p.category_id)::numeric, 2) AS promedio_categoria
FROM products p
JOIN categories cat ON cat.category_id = p.category_id
WHERE p.unit_price = (
  SELECT MAX(p2.unit_price)
  FROM products p2
  WHERE p2.category_id = p.category_id
);
```

## 16. CTE / WITH (expresiones de tabla común)

**Explicación:** WITH nombre AS (consulta) define una subconsulta con nombre que se puede reutilizar como si fuera una tabla en el resto de la consulta principal. Se pueden encadenar

varias CTE (cada una puede usar las anteriores), lo que permite leer la lógica de arriba a abajo en pasos, en vez de anidar subconsultas.

#### Sintaxis básica:

```
WITH cte1 AS (  
    SELECT ...  
)  
cte2 AS (  
    SELECT ... FROM cte1 ...  
)  
SELECT ... FROM cte2;
```

#### Ejemplo práctico:

```
WITH facturacion_cliente AS (  
    SELECT c.customer_id, c.company_name,  
           SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS total  
    FROM customers c  
    JOIN orders o USING (customer_id)  
    JOIN order_details od USING (order_id)  
    GROUP BY c.customer_id, c.company_name  
)  
segmentado AS (  
    SELECT *,  
           NTILE(4) OVER (ORDER BY total DESC) AS cuartil  
    FROM facturacion_cliente  
)  
SELECT  
    CASE cuartil  
        WHEN 1 THEN 'A - Estratégico'  
        WHEN 2 THEN 'B - Consolidado'  
        WHEN 3 THEN 'C - Ocasional'  
        ELSE 'D - Marginal'  
    END AS segmento,  
    COUNT(*) AS num_clientes,  
    ROUND(SUM(total), 2) AS facturacion_segmento,  
    ROUND(100 * SUM(total) / SUM(SUM(total)) OVER (), 2) AS porcentaje_sobre_total  
FROM segmentado  
GROUP BY cuartil  
ORDER BY cuartil;
```

---

## 17. CASE WHEN (condicional en SQL)

**Explicación:** Permite devolver un valor distinto según se cumplan una o varias condiciones, evaluadas en orden hasta encontrar la primera verdadera. Se usa dentro de SELECT (para etiquetar filas) o dentro de una función de agregación (para pivotar datos).

#### Sintaxis básica:

```
CASE  
    WHEN condicion1 THEN valor1  
    WHEN condicion2 THEN valor2  
    ELSE valor_por_defecto  
END
```

#### Ejemplo práctico:

```

SELECT product_name AS producto,
       units_in_stock AS stock,
       reorder_level AS nivel_reposicion,
       units_on_order AS pedido_a_proveedor,
       CASE
         WHEN units_in_stock = 0 THEN 'CRÍTICO'
         ELSE 'AVISO'
       END AS situacion
FROM products
WHERE discontinued = 0
   AND units_in_stock <= reorder_level;

```

---

## 18. NTILE() — segmentación en tramos

**Explicación:** Función de ventana que reparte las filas ordenadas en N grupos (cuartiles, deciles...) de tamaño lo más parecido posible, numerados del 1 al N.

**Sintaxis básica:**

```
NTILE(n) OVER (ORDER BY columna DESC)
```

**Ejemplo práctico:**

```

SELECT customer_id, total,
       NTILE(4) OVER (ORDER BY total DESC) AS cuartil
FROM facturacion_cliente;
-- Cuartil 1 = el 25% de clientes con mayor facturación

```

---

## 19. RANK(), DENSE\_RANK(), ROW\_NUMBER(), PARTITION BY

**Explicación:** Funciones de ventana que numeran filas según un orden. ROW\_NUMBER() da números consecutivos únicos (1,2,3...) sin tener en cuenta empates. RANK() da el mismo número a los empates pero **deja huecos** después (1,2,2,4). DENSE\_RANK() también empata, pero **no deja huecos** (1,2,2,3). PARTITION BY reinicia el conteo dentro de cada grupo (por ejemplo, dentro de cada categoría). No se puede filtrar directamente por una función de ventana en WHERE; hay que calcularla primero en una CTE/subconsulta y filtrar después.

**Sintaxis básica:**

```
ROW_NUMBER() OVER (PARTITION BY grupo ORDER BY columna DESC) AS posicion
```

**Ejemplo práctico:**

```

WITH ventas_producto AS (
  SELECT cat.category_name AS categoria,
         p.product_name AS producto,
         SUM(od.quantity) AS unidades,
         SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS facturacion
  FROM order_details od
  JOIN products p USING (product_id)
  JOIN categories cat USING (category_id)
  GROUP BY cat.category_name, p.product_name
),
rankeado AS (

```

```

SELECT *,
       RANK() OVER (PARTITION BY categoria ORDER BY facturacion DESC) AS posicion_en_categoria,
       RANK() OVER (ORDER BY facturacion DESC) AS posicion_global
FROM ventas_producto
)
SELECT categoria, posicion_en_categoria, producto, unidades, ROUND(facturacion, 2) AS facturacion
FROM rankeado
WHERE posicion_en_categoria <= 3
ORDER BY categoria, posicion_en_categoria;

```

## 20. Marcos de ventana (ROWS BETWEEN), acumulados y LAG()

**Explicación:** Una función de ventana con ORDER BY pero sin marco explícito usa por defecto RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (ideal para un acumulado). Para una media móvil hay que declarar el marco explícitamente con ROWS BETWEEN n PRECEDING AND CURRENT ROW. LAG(columna, n) trae el valor de n filas anteriores dentro de la misma ventana (por defecto, la fila inmediatamente anterior); LEAD() hace lo contrario, mirando hacia adelante.

**Sintaxis básica:**

```

SUM(columna) OVER (ORDER BY fecha) -- acumulado
AVG(columna) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) -- media móvil 3
LAG(columna) OVER (ORDER BY fecha) -- valor anterior

```

**Ejemplo práctico:**

```

WITH mensual AS (
  SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
         SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS facturacion
  FROM orders o
  JOIN order_details od USING (order_id)
  WHERE EXTRACT(YEAR FROM o.order_date) = 1997
  GROUP BY DATE_TRUNC('month', o.order_date)
)
SELECT mes,
       ROUND(facturacion, 2) AS facturacion,
       ROUND(SUM(facturacion) OVER (ORDER BY mes), 2) AS acumulado,
       ROUND(AVG(facturacion) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS media_movil,
       ROUND(LAG(facturacion) OVER (ORDER BY mes), 2) AS mes_anterior,
       ROUND(100 * (facturacion - LAG(facturacion) OVER (ORDER BY mes)) / LAG(facturacion) OVER (ORDER BY mes), 2) AS cambio_mes
FROM mensual
ORDER BY mes;

```

## 21. Pivotado manual (FILTER / CASE WHEN) y ROLLUP

**Explicación:** Pivotar es convertir filas en columnas. En SQL se hace con una función de agregación que solo suma cuando se cumple una condición: la forma estándar es SUM(CASE WHEN condicion THEN valor ELSE 0 END); PostgreSQL ofrece además la cláusula FILTER, más legible: SUM(valor) FILTER (WHERE condicion). ROLLUP genera automáticamente filas de subtotal/total añadidas al resultado agrupado.

**Sintaxis básica:**

```
SUM(importe) FILTER (WHERE anio = 1997) AS f_1997
...
GROUP BY ROLLUP (categoria)
```

### Ejemplo práctico:

```
SELECT
    COALESCE(cat.category_name, 'TOTAL GENERAL') AS categoria,
    ROUND(SUM(importe) FILTER (WHERE anio = 1996), 2) AS f_1996,
    ROUND(SUM(importe) FILTER (WHERE anio = 1997), 2) AS f_1997,
    ROUND(SUM(importe) FILTER (WHERE anio = 1998), 2) AS f_1998,
    ROUND(SUM(importe), 2) AS total,
    CASE
        WHEN SUM(importe) FILTER (WHERE anio = 1998) > SUM(importe) FILTER (WHERE anio = 1997) THEN 'AUMENTA'
        WHEN SUM(importe) FILTER (WHERE anio = 1998) < SUM(importe) FILTER (WHERE anio = 1997) THEN 'DISMINUYE'
        ELSE 'IGUAL'
    END AS tendencia
FROM (
    SELECT cat.category_name,
           EXTRACT(YEAR FROM o.order_date) AS anio,
           od.unit_price::numeric * od.quantity * (1 - od.discount::numeric) AS importe
    FROM order_details od
    JOIN orders o USING (order_id)
    JOIN products p USING (product_id)
    JOIN categories cat USING (category_id)
) datos
RIGHT JOIN categories cat ON cat.category_name = datos.category_name
GROUP BY ROLLUP (cat.category_name)
ORDER BY cat.category_name NULLS LAST;
-- Nota: 1996 solo tiene medio año de datos y 1998 llega hasta mayo,
-- por lo que la comparación 1997 vs 1998 no es directamente comparable.
```

## 22. Funciones de texto

**Explicación:** UPPER()/LOWER() cambian mayúsculas/minúsculas. || (o CONCAT()) concatena cadenas.

### Sintaxis básica:

```
UPPER(columna)
columna1 || ' ' || columna2
CONCAT(columna1, ' ', columna2)
```

### Ejemplo práctico:

```
SELECT UPPER(contact_name) AS contacto_mayus,
       first_name || ' ' || last_name AS nombre_completo
FROM employees;
```

## 23. Funciones de fecha (EXTRACT, DATE\_TRUNC)

**Explicación:** EXTRACT(campo FROM fecha) obtiene una parte de una fecha (año, mes, día...). DATE\_TRUNC('unidad', fecha) “trunca” una fecha a la unidad indicada (por ejemplo, al primer día del mes), muy útil para agrupar por mes o año.

### Sintaxis básica:

```
EXTRACT(YEAR FROM columna_fecha)
DATE_TRUNC('month', columna_fecha)
```

### Ejemplo práctico:

```
SELECT DATE_TRUNC('month', order_date)::date AS mes,
       EXTRACT(YEAR FROM order_date) AS anio,
       COUNT(*) AS num_pedidos
FROM orders
GROUP BY DATE_TRUNC('month', order_date), EXTRACT(YEAR FROM order_date)
ORDER BY mes;
```

## 24. Cast a numeric en cálculos monetarios

**Explicación:** unit\_price, discount y freight son de tipo real (coma flotante), que acumula errores de redondeo al sumar muchas filas. Castear a numeric con ::numeric antes de operar evita esos errores y es obligatorio en toda la práctica para que los resultados coincidan entre compañeros.

### Sintaxis básica:

```
columna::numeric
```

### Ejemplo práctico:

```
SELECT ROUND(SUM(unit_price::numeric * quantity * (1 - discount::numeric)), 2) AS facturacion_tot
FROM order_details;
```

## 25. Checklist rápido de técnicas por pregunta

| Pregunta(s) | Técnicas clave                                          |
|-------------|---------------------------------------------------------|
| 1           | WHERE, BETWEEN, ROUND()                                 |
| 2           | GROUP BY, HAVING, COUNT(DISTINCT)                       |
| 3           | CASE WHEN, comparación entre columnas, cuidado con NULL |
| 4, 5, 6     | INNER JOIN, alias, USING, GROUP BY+HAVING               |
| 7           | LEFT JOIN, COUNT(columna) vs COUNT(*), COALESCE         |
| 8           | SELF JOIN, alias obligatorios                           |
| 9           | CROSS JOIN + LEFT JOIN (rejilla sin huecos)             |
| 10          | FULL JOIN, COALESCE, CASE WHEN                          |
| 11          | UNION ALL                                               |
| 12          | EXCEPT, INTERSECT                                       |
| 13          | NOT EXISTS (anti join), subconsulta correlacionada      |
| 14          | subconsulta escalar en WHERE y SELECT                   |
| 15          | subconsulta en FROM, agregación en dos niveles          |
| 16          | subconsulta correlacionada                              |
| 17          | CTE encadenadas, NTILE()                                |
| 18          | RANK()/ROW_NUMBER(), PARTITION BY, CTE para filtrar     |
| 19          | DATE_TRUNC, marcos ROWS BETWEEN, LAG()                  |
| 20          | pivotado con FILTER, ROLLUP                             |

## Ejercicios resueltos

A continuación se resuelven las 20 preguntas de la Parte 2 sobre la base de datos northwind. Cada consulta incluye un comentario inicial que explica qué devuelve, tal como exige el formato de entrega.

### Pregunta 1 — Catálogo comercial activo

```
-- Productos activos con precio entre 10 y 50 €, de mayor a menor precio
SELECT product_name AS producto,
       ROUND(unit_price::numeric, 2) AS precio
FROM products
WHERE discontinued = 0
  AND unit_price BETWEEN 10 AND 50
ORDER BY precio DESC;
```

---

### Pregunta 2 — Concentración geográfica de la cartera

```
-- Países con 5 o más clientes, con su número de clientes y de ciudades distintas
SELECT c.country AS pais,
       COUNT(*) AS num_clientes,
       COUNT(DISTINCT c.city) AS num_ciudades
FROM customers c
GROUP BY c.country
HAVING COUNT(*) >= 5
ORDER BY num_clientes DESC;
```

---

### Pregunta 3 — Alerta de reposición

```
-- Productos activos con stock por debajo (o igual) de su nivel de reposición
SELECT product_name AS producto,
       units_in_stock AS stock,
       reorder_level AS nivel_reposicion,
       units_on_order AS pedido_a_proveedor,
       CASE
         WHEN units_in_stock = 0 THEN 'CRÍTICO'
         ELSE 'AVISO'
       END AS situacion
FROM products
WHERE discontinued = 0
  AND units_in_stock IS NOT NULL
  AND reorder_level IS NOT NULL
  AND units_in_stock <= reorder_level
ORDER BY stock ASC;
```

Se añade IS NOT NULL sobre ambas columnas porque una comparación con NULL no es ni verdadera ni falsa, y esas filas desaparecerían del resultado sin avisar.

---

## Pregunta 4 — Ficha completa de producto

```
-- Productos de proveedores de Italia, Francia o España, con su categoría y proveedor
SELECT p.product_name AS producto,
       cat.category_name AS categoria,
       s.company_name AS proveedor,
       s.country AS pais,
       s.city AS ciudad
FROM products p
JOIN categories cat ON cat.category_id = p.category_id
JOIN suppliers s ON s.supplier_id = p.supplier_id
WHERE s.country IN ('Italy', 'France', 'Spain')
ORDER BY s.country, p.product_name;
```

---

## Pregunta 5 — Detalle valorizado de un pedido

```
-- Líneas del pedido 10248 con su importe, más cliente y fecha del pedido
SELECT c.company_name AS cliente,
       o.order_date AS fecha_pedido,
       p.product_name AS producto,
       od.unit_price AS precio_unitario,
       od.quantity AS cantidad,
       od.discount AS descuento,
       ROUND(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric), 2) AS importe_linea
FROM orders o
JOIN customers c USING (customer_id)
JOIN order_details od USING (order_id)
JOIN products p USING (product_id)
WHERE o.order_id = 10248
ORDER BY p.product_name;
```

---

## Pregunta 6 — Ranking de categorías por facturación

```
-- Categorías con más de 100.000 € de facturación histórica
SELECT cat.category_name AS categoria,
       COUNT(*) AS num_lineas,
       COUNT(DISTINCT p.product_id) AS num_productos,
       ROUND(SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)), 2) AS facturacion
FROM categories cat
JOIN products p ON p.category_id = cat.category_id
JOIN order_details od ON od.product_id = p.product_id
GROUP BY cat.category_name
HAVING SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) > 100000
ORDER BY facturacion DESC;
```

---

## Pregunta 7 — Clientes sin actividad comercial

```
-- Todos los clientes, con su número de pedidos y último pedido (o 'SIN PEDIDOS')
SELECT c.company_name AS cliente,
       c.country AS pais,
       COUNT(o.order_id) AS num_pedidos,
       COALESCE(MAX(o.order_date)::text, 'SIN PEDIDOS') AS ultimo_pedido
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.company_name, c.country
ORDER BY num_pedidos ASC, cliente;
```

---

## Pregunta 8 — Organigrama de la fuerza de ventas

```
-- Empleados con su responsable directo (autounión de employees)
SELECT emp.first_name || ' ' || emp.last_name AS empleado,
       emp.title AS cargo,
       COALESCE(jefe.first_name || ' ' || jefe.last_name, 'DIRECCIÓN GENERAL') AS responsable,
       COALESCE(jefe.title, '') AS cargo_responsable
FROM employees emp
LEFT JOIN employees jefe ON emp.reports_to = jefe.employee_id
ORDER BY empleado;
```

---

## Pregunta 9 — Rejilla de cobertura categoría × año

```
-- Facturación por categoría y año, incluyendo combinaciones sin ventas (0)
WITH anios AS (
    SELECT generate_series(1996, 1998) AS anio
)
SELECT cat.category_name AS categoria,
       a.anio,
       COALESCE(ROUND(SUM(
           od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)
       ), 2), 0) AS facturacion
FROM categories cat
CROSS JOIN anios a
LEFT JOIN products p ON p.category_id = cat.category_id
LEFT JOIN order_details od ON od.product_id = p.product_id
LEFT JOIN orders o ON o.order_id = od.order_id
                     AND EXTRACT(YEAR FROM o.order_date) = a.anio
GROUP BY cat.category_name, a.anio
ORDER BY cat.category_name, a.anio;
```

---

## Pregunta 10 — Mapa de países: clientes frente a proveedores

```
-- Países con presencia de clientes y/o proveedores
WITH clientes_pais AS (
    SELECT country, COUNT(*) AS n FROM customers GROUP BY country
```

```

),
proveedores_pais AS (
    SELECT country, COUNT(*) AS n FROM suppliers GROUP BY country
)
SELECT COALESCE(cp.country, pp.country) AS pais,
       COALESCE(cp.n, 0) AS num_clientes,
       COALESCE(pp.n, 0) AS num_proveedores,
       CASE
           WHEN cp.n IS NOT NULL AND pp.n IS NOT NULL THEN 'AMBOS'
           WHEN cp.n IS NOT NULL THEN 'SOLO CLIENTES'
           ELSE 'SOLO PROVEEDORES'
       END AS tipo_presencia
FROM clientes_pais cp
FULL JOIN proveedores_pais pp ON cp.country = pp.country
ORDER BY pais;

```

---

## Pregunta 11 — Directorio unificado de contactos

```

-- Directorio único de contactos: clientes, proveedores y empleados
SELECT 'CLIENTE' AS origen,
       UPPER(contact_name) AS contacto,
       company_name AS organizacion,
       city AS ciudad,
       country AS pais
FROM customers

UNION ALL

SELECT 'PROVEEDOR',
       UPPER(contact_name),
       company_name,
       city,
       country
FROM suppliers

UNION ALL

SELECT 'EMPLEADO',
       UPPER(first_name || ' ' || last_name),
       'NORTHWIND TRADERS',
       city,
       country
FROM employees

ORDER BY origen, pais;

```

Se usa UNION ALL y no UNION porque, aunque un cliente y un proveedor coincidieran en nombre de contacto y ciudad, ambas filas representan entidades distintas y deben conservarse las dos; UNION las fusionaría en una sola por eliminar duplicados.

---

## Pregunta 12 — Mercados con desequilibrio

```
-- a) Países con clientes pero sin ningún proveedor
SELECT country AS pais
FROM customers
EXCEPT
SELECT country
FROM suppliers
ORDER BY pais;

-- b) Países con clientes y proveedores a la vez
SELECT country AS pais
FROM customers
INTERSECT
SELECT country
FROM suppliers
ORDER BY pais;
```

---

## Pregunta 13 — Clientes que nunca han comprado pescado

```
-- Clientes que nunca han pedido productos de la categoría 'Seafood'
SELECT c.company_name AS cliente,
       c.country AS pais,
       COUNT(o.order_id) AS pedidos_realizados
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o2
    JOIN order_details od ON od.order_id = o2.order_id
    JOIN products p ON p.product_id = od.product_id
    JOIN categories cat ON cat.category_id = p.category_id
    WHERE o2.customer_id = c.customer_id
      AND cat.category_name = 'Seafood'
)
GROUP BY c.company_name, c.country
ORDER BY pedidos_realizados DESC;
```

---

## Pregunta 14 — Productos por encima de la media

```
-- Productos activos con precio superior a la media del catálogo
SELECT product_name AS producto,
       ROUND(unit_price::numeric, 2) AS precio,
       ROUND((SELECT AVG(unit_price) FROM products WHERE discontinued = 0)::numeric, 2) AS precio_promedio,
       ROUND(
           unit_price::numeric - (SELECT AVG(unit_price) FROM products WHERE discontinued = 0)::numeric,
           2) AS diferencia
FROM products
WHERE discontinued = 0
  AND unit_price > (SELECT AVG(unit_price) FROM products WHERE discontinued = 0)
```

ORDER BY diferencia DESC;

---

## Pregunta 15 — Ticket medio por cliente

```
-- Top 15 clientes por ticket medio (importe medio por pedido)
SELECT cliente, pais,
       COUNT(*) AS num_pedidos,
       ROUND(SUM(importe_pedido), 2) AS importe_total,
       ROUND(AVG(importe_pedido), 2) AS ticket_medio
FROM (
  SELECT o.order_id,
         c.company_name AS cliente,
         c.country AS pais,
         SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS importe_pedido
  FROM orders o
  JOIN customers c USING (customer_id)
  JOIN order_details od USING (order_id)
  GROUP BY o.order_id, c.company_name, c.country
) AS pedidos_valorizados
GROUP BY cliente, pais
ORDER BY ticket_medio DESC
LIMIT 15;
```

---

## Pregunta 16 — El producto más caro de cada categoría

```
-- Producto más caro de cada categoría, con el precio medio de su categoría
SELECT cat.category_name AS categoria,
       p.product_name AS producto,
       ROUND(p.unit_price::numeric, 2) AS precio,
       ROUND(
         (SELECT AVG(p2.unit_price) FROM products p2 WHERE p2.category_id = p.category_id)::numeric,
         2) AS precio_medio_categoria
FROM products p
JOIN categories cat ON cat.category_id = p.category_id
WHERE p.unit_price = (
  SELECT MAX(p2.unit_price)
  FROM products p2
  WHERE p2.category_id = p.category_id
)
ORDER BY categoria;
```

---

## Pregunta 17 — Segmentación ABC de la cartera de clientes

```
-- Segmentación de clientes en cuartiles (A/B/C/D) por facturación
WITH facturacion_cliente AS (
  SELECT c.customer_id, c.company_name,
         SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS total
  FROM customers c
  JOIN order_details od ON od.customer_id = c.customer_id
  JOIN orders o ON o.order_id = od.order_id
)
```

```

    JOIN orders o USING (customer_id)
    JOIN order_details od USING (order_id)
    GROUP BY c.customer_id, c.company_name
),
segmentado AS (
    SELECT *,
           NTILE(4) OVER (ORDER BY total DESC) AS cuartil
    FROM facturacion_cliente
)
SELECT
    CASE cuartil
        WHEN 1 THEN 'A - Estratégico'
        WHEN 2 THEN 'B - Consolidado'
        WHEN 3 THEN 'C - Ocasional'
        ELSE 'D - Marginal'
    END AS segmento,
    COUNT(*) AS num_clientes,
    ROUND(SUM(total), 2) AS facturacion_segmento,
    ROUND(100 * SUM(total) / SUM(SUM(total)) OVER (), 2) AS porcentaje_sobre_total
FROM segmentado
GROUP BY cuartil
ORDER BY cuartil;

```

---

## Pregunta 18 — Los tres productos más vendidos de cada categoría

```

-- Top 3 productos por facturación en cada categoría, con su posición global
WITH ventas_producto AS (
    SELECT cat.category_name AS categoria,
           p.product_name AS producto,
           SUM(od.quantity) AS unidades,
           SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS facturacion
    FROM order_details od
    JOIN products p USING (product_id)
    JOIN categories cat USING (category_id)
    GROUP BY cat.category_name, p.product_name
),
rankeado AS (
    SELECT *,
           RANK() OVER (PARTITION BY categoria ORDER BY facturacion DESC) AS posicion_en_categoria,
           RANK() OVER (ORDER BY facturacion DESC) AS posicion_global
    FROM ventas_producto
)
SELECT categoria, posicion_en_categoria, producto, unidades,
       ROUND(facturacion, 2) AS facturacion, posicion_global
FROM rankeado
WHERE posicion_en_categoria <= 3
ORDER BY categoria, posicion_en_categoria;

```

## Pregunta 19 — Evolución mensual con acumulado y media móvil

```
-- Evolución mensual de 1997: facturación, acumulado, media móvil 3m y variación
WITH mensual AS (
    SELECT DATE_TRUNC('month', o.order_date)::date AS mes,
           SUM(od.unit_price::numeric * od.quantity * (1 - od.discount::numeric)) AS facturacion
    FROM orders o
    JOIN order_details od USING (order_id)
    WHERE EXTRACT(YEAR FROM o.order_date) = 1997
    GROUP BY DATE_TRUNC('month', o.order_date)
)
SELECT mes,
       ROUND(facturacion, 2) AS facturacion,
       ROUND(SUM(facturacion) OVER (ORDER BY mes), 2) AS acumulado,
       ROUND(AVG(facturacion) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS media_movil_3m,
       ROUND(LAG(facturacion) OVER (ORDER BY mes), 2) AS mes_anterior,
       ROUND(
           100 * (facturacion - LAG(facturacion) OVER (ORDER BY mes))
           / NULLIF(LAG(facturacion) OVER (ORDER BY mes), 0),
           2) AS variacion_pct
FROM mensual
ORDER BY mes;
```

El primer mes no tiene mes anterior, así que mes\_anterior y variacion\_pct salen NULL en esa fila; es el comportamiento esperado y se documenta como tal. Se usa NULLIF para evitar una división por cero si algún mes anterior tuviera facturación 0.

## Pregunta 20 — Cuadro de mando anual por categoría

```
-- Facturación por categoría y año en columnas, con fila de totales y tendencia
-- NOTA: 1996 solo tiene datos desde julio y 1998 solo hasta mayo, por lo que
-- la comparación de tendencia 1997 vs 1998 no es directamente comparable
-- (son periodos de longitud distinta); se deja constancia de ello aquí.
SELECT
    COALESCE(cat.category_name, 'TOTAL GENERAL') AS categoria,
    ROUND(SUM(importe) FILTER (WHERE anio = 1996), 2) AS f_1996,
    ROUND(SUM(importe) FILTER (WHERE anio = 1997), 2) AS f_1997,
    ROUND(SUM(importe) FILTER (WHERE anio = 1998), 2) AS f_1998,
    ROUND(SUM(importe), 2) AS total,
    ROUND(100 * SUM(importe) / SUM(SUM(importe)) OVER (), 2) AS peso_pct,
    CASE
        WHEN SUM(importe) FILTER (WHERE anio = 1998) > SUM(importe) FILTER (WHERE anio = 1997) THEN 'AUMENTA'
        WHEN SUM(importe) FILTER (WHERE anio = 1998) < SUM(importe) FILTER (WHERE anio = 1997) THEN 'DISMINUYE'
        ELSE 'IGUAL'
    END AS tendencia
FROM (
    SELECT cat.category_name,
           EXTRACT(YEAR FROM o.order_date) AS anio,
           od.unit_price::numeric * od.quantity * (1 - od.discount::numeric) AS importe
    FROM order_details od
    JOIN orders o USING (order_id)
    JOIN products p USING (product_id)
    JOIN categories cat USING (category_id)
```

```
) datos  
RIGHT JOIN categories cat ON cat.category_name = datos.category_name  
GROUP BY ROLLUP (cat.category_name)  
ORDER BY cat.category_name NULLS LAST;
```